

AMERICAN INTERNATIONAL UNIVERSITY BANGLADESH (AIUB)

| FACULTY OF SCIENCE & TECHNOLOGY |
| DEPARTMENT OF COMPUTER SCIENCE |

Introduction to Data Science
[Mid term project - solo]
Summer 2025-2026

Section: D
ID No: 22-48676-3

Submitted By:
Foysal Ahmed (Neloy)

Submitted To:
Kamrun Naher Koli
| Faculty, Department of Data Science |

Date of Submission: 27 July 2026

Dataset:

Student Performance & Study Habits Dataset

Source: Kaggle
Link: https://www.kaggle.com/datasets/harshadapatil31/student-performance-and-study-habits-dataset

Dataset Overview:

This dataset is designed for student performance prediction and combines study habits, lifestyle factors, and demographic information with academic outcomes. It contains 1000 samples across 12 attributes, providing a solid foundation for exploratory analysis and predictive modeling.

Attributes:

1. **student_id:** Unique identifier for each student.
2. **gender:** Categorical variable - Male / Female.
3. **study_time_hours:** Continuous variable representing average daily study time (hours).
4. **attendance_percent:** Continuous variable measuring class attendance percentage.
5. **sleep_hours:** Continuous variable representing average daily sleep (hours).
6. **parental_education:** Categorical variable indicating the highest education level of a parent.
7. **internet_access:** Binary categorical variable (Yes/No) indicating internet access.
8. **extracurricular_activities:** Binary categorical variable (Yes/No) indicating participation.
9. **part_time_job:** Binary categorical variable (Yes/No) indicating employment status.
10. **previous_grade:** Continuous variable - grade from the previous term/exam (0–100).
11. **final_exam_score:** Continuous variable - final exam score (0–100).
12. **final_grade:** Ordinal categorical variable - letter grade (A, B, C, D, F).

Install and Importing Library

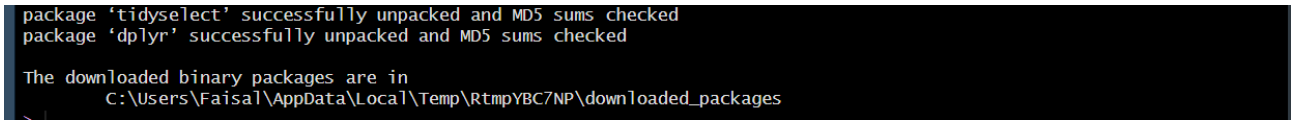
This step installs and imports the necessary library to the Rstudio.

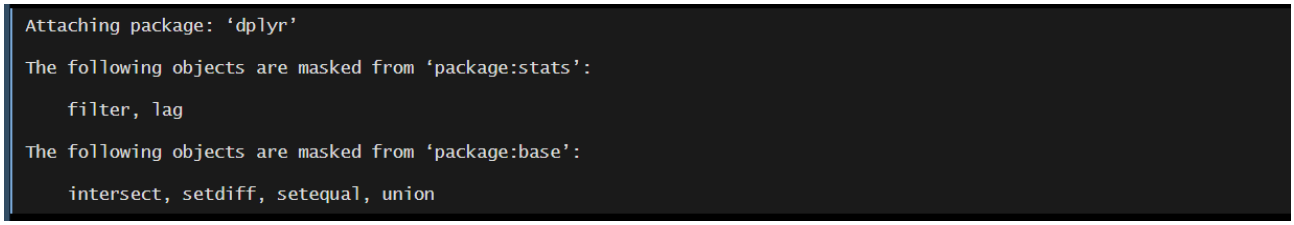
Code:





Output:

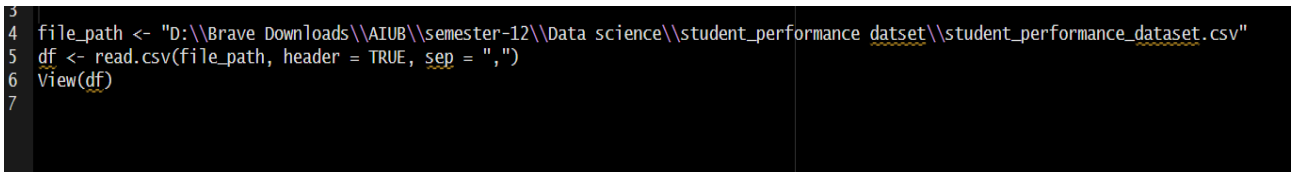




Importing and Viewing the Dataset

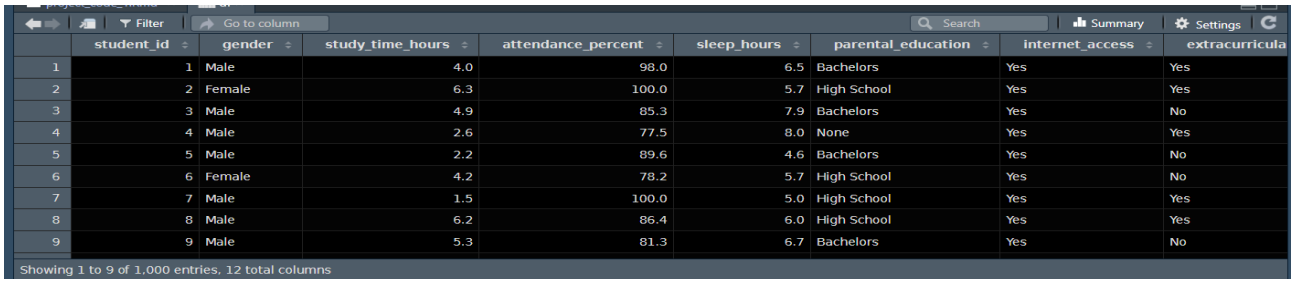
This step loads the CSV file into R, ensuring proper column headers and delimiter settings, then opens it for inspection in RStudio's data viewer.

Code:



This code imports a CSV file from the specified path and assigns it to the variable **df.** The header = TRUE argument indicates that the first row of the CSV contains column names, while sep = "," specifies that the file uses commas as delimiters. The **View(df)** command displays the imported dataset in a tabular format in RStudio's data viewer for easy inspection

Output:



Filter | Go to column | Search | Summary | Settings

	student_id	gender	study_time_hours	attendance_percent	sleep_hours	parental_education	internet_access	extracurricula
1	1	Male	4.0	98.0	6.5	Bachelors	Yes	Yes
2	2	Female	6.3	100.0	5.7	High School	Yes	Yes
3	3	Male	4.9	85.3	7.9	Bachelors	Yes	No
4	4	Male	2.6	77.5	8.0	None	Yes	Yes
5	5	Male	2.2	89.6	4.6	Bachelors	Yes	No
6	6	Female	4.2	78.2	5.7	High School	Yes	No
7	7	Male	1.5	100.0	5.0	High School	Yes	Yes
8	8	Male	6.2	86.4	6.0	High School	Yes	Yes
9	9	Male	5.3	81.3	6.7	Bachelors	Yes	No

Showing 1 to 9 of 1,000 entries, 12 total columns

Examining the Dataset Dimensions

Code:

```
#numbers of column and row
cat("Number of rows:", nrow(df), "\n")
cat("Number of columns:", ncol(df))
```

Output:

```
> cat("Number of rows:", nrow(df), "\n")
Number of rows: 1000
> cat("Number of columns:", ncol(df))
Number of columns: 12
>
```

This code outputs 1000 rows and 12 attribute columns for the dataset.

Inspecting Dataset Structure

Examining the structure and attributes of the dataset.

Code:

```
#finding structure
str(df)
```

13:8 finding structure

Output:

```
> str(df)
'data.frame':   1000 obs. of  12 variables:
 $ student_id               : int  1 2 3 4 5 6 7 8 9 10 ...
 $ gender                   : chr  "Male" "Female" "Male" "Male" ...
 $ study_time_hours         : num  4 6.3 4.9 2.6 2.2 4.2 1.5 6.2 5.3 2.8 ...
 $ attendance_percent       : num  98 100 85.3 77.5 89.6 78.2 100 86.4 81.3 86.8 ...
 $ sleep_hours              : num  6.5 5.7 7.9 8 4.6 5.7 5 6 6.7 5.1 ...
 $ parental_education       : chr  "Bachelors" "High School" "Bachelors" "None" ...
 $ internet_access          : chr  "Yes" "Yes" "Yes" "Yes" ...
 $ extracurricular_activities: chr  "Yes" "Yes" "No" "Yes" ...
 $ part_time_job            : chr  "No" "Yes" "Yes" "No" ...
 $ previous_grade           : num  76.9 75.5 88.5 85.1 61.8 79.8 58.6 78.4 63.8 57.7 ...
 $ final_exam_score         : num  100 100 97.3 83.8 68.3 81.6 69.6 88 84.4 74.3 ...
 $ final_grade              : chr  "A" "A" "A" "B" ...
```

Handling Missing Values

Code:

```
#finding missing values per culmn
df[df == ""] <- NA
num_of_missing_value_per_column <- sapply(df, function(x) sum(is.na(x)))
print(num_of_missing_value_per_column)
print(paste("Total missing values:", sum(is.na(df))))
```

Output:

```
> num_of_missing_value_per_column <- sapply(df, function(x) sum(is.na(x)))
> print(num_of_missing_value_per_column)
            student_id                 gender        study_time_hours       attendance_percent
                     0                      0                      0                      0
           sleep_hours      parental_education       internet_access extracurricular_activities
                     0                      0                      0                      0
          part_time_job         previous_grade       final_exam_score            final_grade
                     0                      0                      0                      0
> print(paste("Total missing values:", sum(is.na(df))))
[1] "Total missing values: 0"
>
```

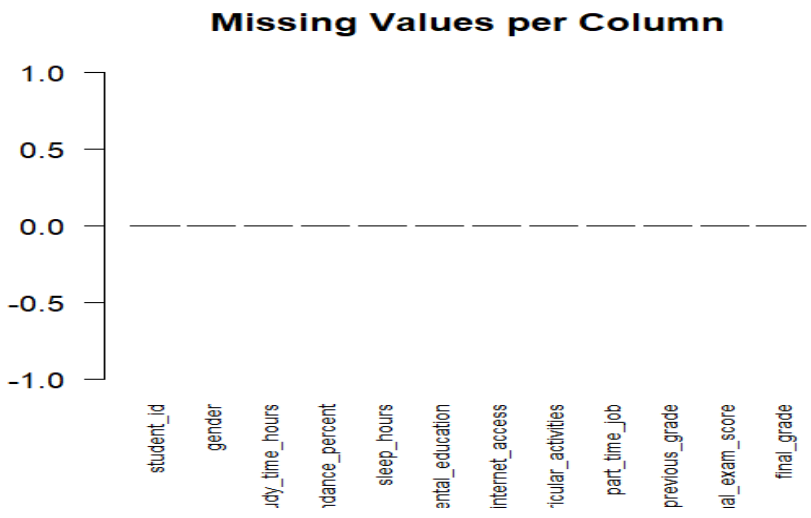
The output confirms that this dataset has no missing values across any of the 12 columns. Since the data is already complete, no imputation using mean or mode was necessary, and the dataset moved directly to the converting and outlier-handling stages.

Plot for missing values

Code:

```
#barplot of missing values
barplot(num_of_missing_value_per_column,
        main = "Missing Values per Column",
        col = "cyan",
        las = 2,
        cex.names = 0.7)
```

Output:



Converting Categorical Values into Numerical Values

Code:

```
#converting categories to numerical 0,1,2 etc.
df$gender <- ifelse(df$gender == "Male", 1, 0)
df$internet_access <- ifelse(df$internet_access == "Yes", 1, 0)
df$extracurricular_activities <- ifelse(df$extracurricular_activities == "Yes", 1, 0)
df$part_time_job <- ifelse(df$part_time_job == "Yes", 1, 0)

# final_grade is ordinal (A > B > C > D > F), so i am mapping it on a numeric scale - decending
grade_map <- c(A = 4, B = 3, C = 2, D = 1, F = 0)
df$final_grade_numeric <- grade_map[df$final_grade]

str(df)
```

Output:

```
> df$gender <- ifelse(df$gender == "Male", 1, 0)
> df$internet_access <- ifelse(df$internet_access == "Yes", 1, 0)
> df$extracurricular_activities <- ifelse(df$extracurricular_activities == "Yes", 1, 0)
> df$part_time_job <- ifelse(df$part_time_job == "Yes", 1, 0)
> grade_map <- c(A = 4, B = 3, C = 2, D = 1, F = 0)
> df$final_grade_numeric <- grade_map[df$final_grade]
> str(df)
'data.frame':   1000 obs. of  13 variables:
 $ student_id               : int  1 2 3 4 5 6 7 8 9 10 ...
 $ gender                   : num  1 0 1 1 1 0 1 1 1 0 ...
 $ study_time_hours         : num  4 6.3 4.9 2.6 2.2 4.2 1.5 6.2 5.3 2.8 ...
 $ attendance_percent       : num  98 100 85.3 77.5 89.6 78.2 100 86.4 81.3 86.8 ...
 $ sleep_hours              : num  6.5 5.7 7.9 8 4.6 5.7 5 6 6.7 5.1 ...
 $ parental_education       : chr  "Bachelors" "High School" "Bachelors" "None" ...
 $ internet_access          : num  1 1 1 1 1 1 1 1 1 1 ...
 $ extracurricular_activities: num  1 1 0 1 0 0 1 1 0 1 ...
 $ part_time_job            : num  0 1 1 0 1 0 1 0 0 0 ...
 $ previous_grade           : num  76.9 75.5 88.5 85.1 61.8 79.8 58.6 78.4 63.8 57.7 ...
 $ final_exam_score         : num  100 100 97.3 83.8 68.3 81.6 69.6 88 84.4 74.3 ...
 $ final_grade              : chr  "A" "A" "A" "B" ...
 $ final_grade_numeric      : num  4 4 4 3 1 3 1 3 3 2 ...
```

The binary categorical columns like, **gender**, **internet_access**, **extracurricular_activities**, and **part_time_job** - were converted from character to numeric (1/0) using **ifelse()**. The ordinal **final_grade** column was mapped onto a 0–4 numeric scale using a named vector, preserving the natural ranking of A through F rather than treating the grades as unrelated categories. The **str(df)** output confirmed all encoded columns now appear as num type, while **parental_education** and **final_grade** remain as character columns since they were not encoded at this stage.

Finding Outlier

Code:

```
#finding outliers
find_outliers <- function(column) {
  Q1 <- quantile(column, 0.25, na.rm = TRUE)
  Q3 <- quantile(column, 0.75, na.rm = TRUE)
  IQR_value <- Q3 - Q1
  lower_bound <- Q1 - 1.5 * IQR_value
  upper_bound <- Q3 + 1.5 * IQR_value
  outlier_indices <- which(column < lower_bound | column > upper_bound)
  return(outlier_indices)
}

outlier_indices_study     <- find_outliers(df$study_time_hours)
outlier_indices_attend    <- find_outliers(df$attendance_percent)
outlier_indices_sleep     <- find_outliers(df$sleep_hours)
outlier_indices_prevgrade <- find_outliers(df$previous_grade)
outlier_indices_finalexam <- find_outliers(df$final_exam_score)

print(outlier_indices_study)
print(outlier_indices_attend)
print(outlier_indices_sleep)
print(outlier_indices_prevgrade)
print(outlier_indices_finalexam)
```

Output:

```
> outlier_indices_finalexam <- find_outliers(df$final_exam_score)
>
> print(outlier_indices_study)
[1]  93 269 370 848
> print(outlier_indices_attend)
[1] 154 206 365 586 639 873
> print(outlier_indices_sleep)
[1] 562 767
> print(outlier_indices_prevgrade)
[1] 611 761 776
> print(outlier_indices_finalexam)
[1] 148
>
```

Outliers were detected using the Interquartile Range **(IQR)** method, which flags values falling below **Q1 − 1.5×IQR** or above **Q3 + 1.5×IQR**. This method was applied only to the continuous numeric variables; binary columns like **gender** and **internet_access** were excluded, as IQR-based detection is not meaningful for categorical or binary data.

Removing Outliers

Code:

```
#outlier removing
all_outlier_indices <- unique(c(outlier_indices_study,
                                outlier_indices_attend,
                                outlier_indices_sleep,
                                outlier_indices_prevgrade,
                                outlier_indices_finalexam))
df <- df[-all_outlier_indices, ]
```

A combined list of all outlier indices from the five continuous columns was created using **c()**. The **unique()** function ensured that any row flagged in

more than one column was not removed twice. The dataset was then updated to exclude all 16 flagged rows, effectively removing the identified outliers.

Output:

```
[1] 148
> all_outlier_indices <- unique(c(outlier_indices_study,
+                                 outlier_indices_attend,
+                                 outlier_indices_sleep,
+                                 outlier_indices_prevgrade,
+                                 outlier_indices_finalexam))
> df <- df[-all_outlier_indices, ]
>
```

Data Filtering

Code:

```
#data filtering
filtered_data <- df[df$attendance_percent >= 0 & df$attendance_percent <= 100 &
                    df$study_time_hours >= 0 & df$study_time_hours <= 12, ]
View(head(filtered_data))

```

A filter was applied to include only rows where **attendance_percent** falls within a valid 0–100% range and **study_time_hours** falls within a realistic 0–12 hour range. This step acts as an additional sanity check beyond outlier removal, ensuring the data meets logically valid bounds before analysis.

Output:

project_code_1.Rmd* | head(filtered_data) | df

Filter | Go to column | Search | Summary | Settings

	student_id	gender	study_time_hours	attendance_percent	sleep_hours	parental_edu
1	1	1	4.0	98.0	6.5	Bachelors
2	2	0	6.3	100.0	5.7	High School
3	3	1	4.9	85.3	7.9	Bachelors
4	4	1	2.6	77.5	8.0	None
5	5	1	2.2	89.6	4.6	Bachelors
6	6	0	4.2	78.2	5.7	High School

13 columns

student_id <int>
[1, 6] 0% NA

gender <dbl>
[0, 1] 0% NA

Normalizing the Dataset

Code:

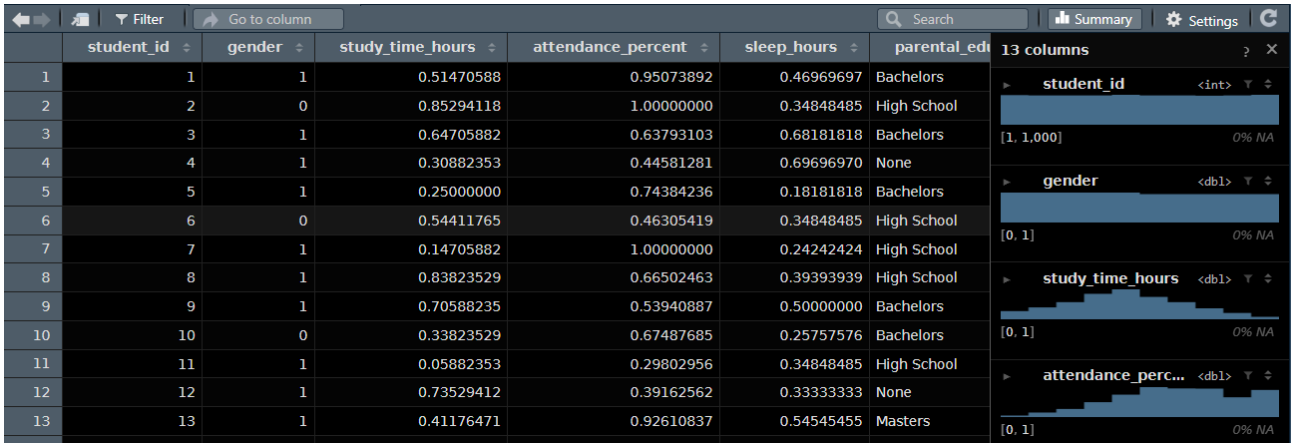
```
#normalizing the dataset
normalize <- function(x) {
  return((x - min(x)) / (max(x) - min(x)))
}

normalize_dataset <- df
normalize_dataset$study_time_hours  <- normalize(df$study_time_hours)
normalize_dataset$attendance_percent <- normalize(df$attendance_percent)
normalize_dataset$sleep_hours        <- normalize(df$sleep_hours)
normalize_dataset$previous_grade     <- normalize(df$previous_grade)
normalize_dataset$final_exam_score   <- normalize(df$final_exam_score)

View(normalize_dataset)
```

The dataset **df** was copied into **normalize_dataset** to preserve the original data. Min-max normalization was applied to the five continuous variables, scaling their values to a range between 0 and 1 using **(x–min(x))/(max(x)-min(x))**. This ensures all continuous variables are on a comparable scale before further analysis.

Output:



	student_id	gender	study_time_hours	attendance_percent	sleep_hours	parental_edu
1	1	1	0.51470588	0.95073892	0.46969697	Bachelors
2	2	0	0.85294118	1.00000000	0.34848485	High School
3	3	1	0.64705882	0.63793103	0.68181818	Bachelors
4	4	1	0.30882353	0.44581281	0.69696970	None
5	5	1	0.25000000	0.74384236	0.18181818	Bachelors
6	6	0	0.54411765	0.46305419	0.34848485	High School
7	7	1	0.14705882	1.00000000	0.24242424	High School
8	8	1	0.83823529	0.66502463	0.39393939	High School
9	9	1	0.70588235	0.53940887	0.50000000	Bachelors
10	10	0	0.33823529	0.67487685	0.25757576	Bachelors
11	11	1	0.05882353	0.29802956	0.34848485	High School
12	12	1	0.73529412	0.39162562	0.33333333	None
13	13	1	0.41176471	0.92610837	0.54545455	Masters

13 columns

student_id <int> [1, 1,000] 0% NA

gender <dbl> [0, 1] 0% NA

study_time_hours <dbl> [0, 1] 0% NA

attendance_perc... <dbl> [0, 1] 0% NA

Balancing the Dataset

1. Finding the Class Distribution

Code:

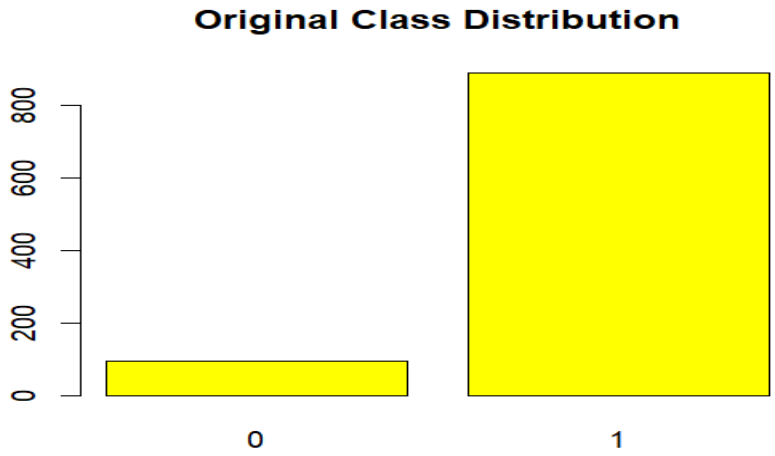
```
#balancing

idata <- normalize_dataset
idata$pass_fail <- ifelse(idata$final_grade %in% c("A", "B", "C"), 1, 0)  # 1 = Pass, 0 = Fail

class_dist <- table(idata$pass_fail)
class_dist
barplot(class_dist, main = "Original Class Distribution", col = "yellow")
```

A binary target variable **pass_fail** was derived from **final_grade** (A/B/C = Pass, D/F = Fail). The result shows a strongly imbalanced dataset, with only 96 Fail cases against 888 Pass cases, which could bias any predictive model toward the majority class. (D is considered as fail for butifying purpose)

Output:



2.Balancing Using Oversampling

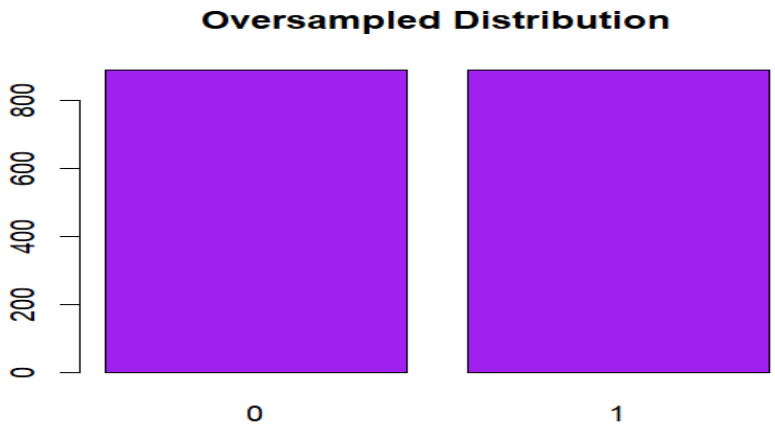
Code:

```
#oversampling
library(dplyr)
if (class_dist[1] > class_dist[2]) {
  majority <- filter(idata, pass_fail == 0)
  minority <- filter(idata, pass_fail == 1)
} else {
  majority <- filter(idata, pass_fail == 1)
  minority <- filter(idata, pass_fail == 0)
}

set.seed(100)
os_minority <- minority %>% sample_n(nrow(majority), replace = TRUE)
os_balanced_data <- bind_rows(majority, os_minority)
table(os_balanced_data$pass_fail)
barplot(table(os_balanced_data$pass_fail), main = "Oversampled Distribution", col = "purple")
```

The majority (Pass) and minority (Fail) classes were identified, and the minority class was resampled with replacement using **sample_n()** until it matched the majority class size. The two groups were combined into **os_balanced_data**, resulting in a perfectly balanced 888/888 split.

Output:



3. Balancing Using Undersampling

Code:

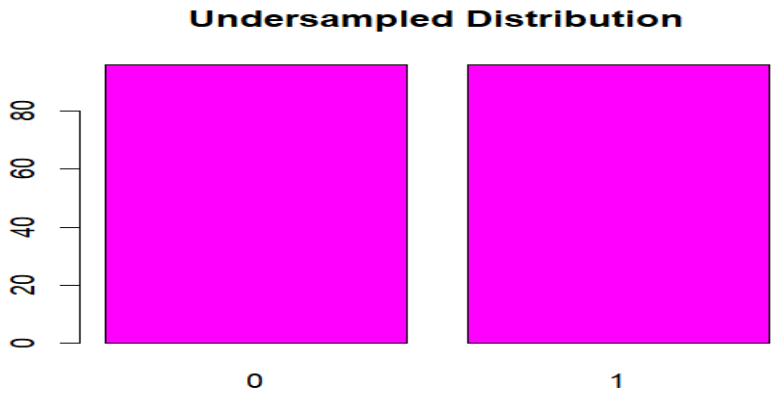
```
#undersampling

us_majority <- majority %>% sample_n(nrow(minority))
us_balanced_data <- bind_rows(minority, us_majority)
table(us_balanced_data$pass_fail)
barplot(table(us_balanced_data$pass_fail), main = "Undersampled Distribution", col = "magenta")

```

As an alternative, the majority class was randomly downsampled (without replacement) to match the minority class size, producing a balanced 96/96 dataset. This retains only original, non-duplicated rows but discards a large portion of majority-class data.

Output:



Splitting the Dataset

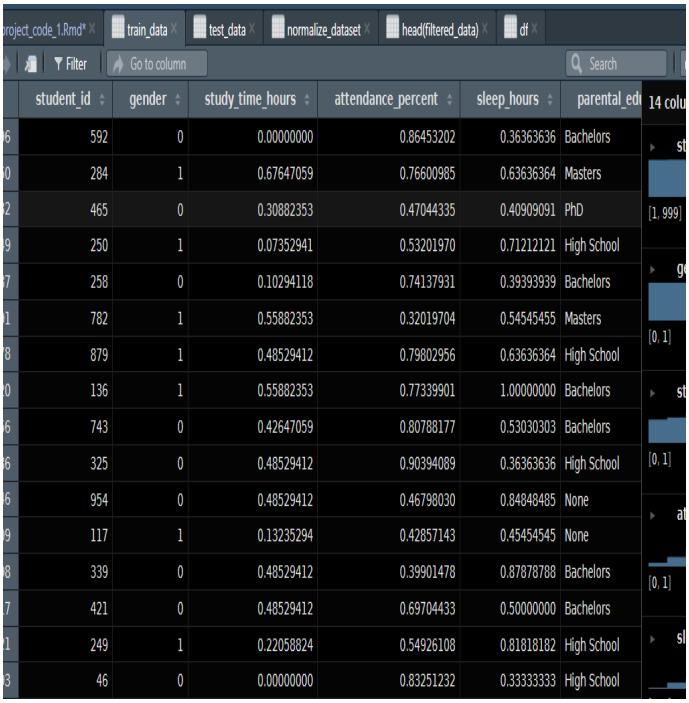
Code:

Output:

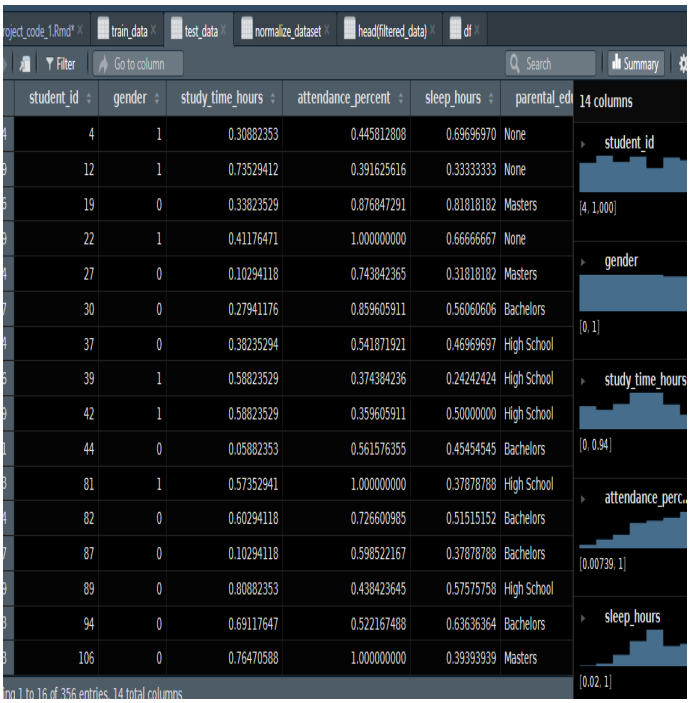
```
#spilitting

final_data <- os_balanced_data  # or us_balanced_data, your choice

n <- nrow(final_data)
random_index <- sample(1:n, size = 0.8 * n)
train_data <- final_data[random_index, ]
test_data  <- final_data[-random_index, ]
View(train_data)
View(test_data)
```



student_id	gender	study_time_hours	attendance_percent	sleep_hours	parental_edu
592	0	0.00000000	0.86453202	0.36363636	Bachelors
284	1	0.67647059	0.76600985	0.63636364	Masters
465	0	0.30882353	0.47044335	0.40909091	PhD
250	1	0.07352941	0.53201970	0.71212121	High School
258	0	0.10294118	0.74137931	0.39393939	Bachelors
782	1	0.55882353	0.32019704	0.54545455	Masters
879	1	0.48529412	0.79802956	0.63636364	High School
136	1	0.55882353	0.77339901	1.00000000	Bachelors
743	0	0.42647059	0.80788177	0.53030303	Bachelors
325	0	0.48529412	0.90394089	0.36363636	High School
954	0	0.48529412	0.46798030	0.84848485	None
117	1	0.13235294	0.42857143	0.45454545	None
339	0	0.48529412	0.39901478	0.87878788	Bachelors
421	0	0.48529412	0.69704433	0.50000000	Bachelors
249	1	0.22058824	0.54926108	0.81818182	High School
46	0	0.00000000	0.83251232	0.33333333	High School



student_id	gender	study_time_hours	attendance_percent	sleep_hours	parental_edu
4	1	0.30882353	0.445812808	0.69696970	None
12	1	0.73529412	0.391625616	0.33333333	None
19	0	0.33823529	0.876847291	0.81818182	Masters
22	1	0.41176471	1.000000000	0.66666667	None
27	0	0.10294118	0.743842365	0.31818182	Masters
30	0	0.27941176	0.859605911	0.56060606	Bachelors
37	0	0.38235294	0.541871921	0.46969697	High School
39	1	0.58823529	0.374384236	0.24242424	High School
42	1	0.58823529	0.359605911	0.50000000	High School
44	0	0.05882353	0.561576355	0.45454545	Bachelors
81	1	0.57352941	1.000000000	0.37878788	High School
82	0	0.60294118	0.726600985	0.51515152	Bachelors
87	0	0.10294118	0.598522167	0.37878788	Bachelors
89	0	0.80882353	0.438423645	0.57575758	High School
94	0	0.69117647	0.522167488	0.63636364	Bachelors
106	0	0.76470588	1.000000000	0.39393939	Masters

The oversampled dataset was chosen as **final_data** since it retains more total data than the undersampled version. An 80/20 split was applied using random row indices: 80% of rows were assigned to **train_data** and the remaining 20% to **test_data**, ensuring the data is properly divided for future model training and evaluation. The left output is the **train_data** and the right output is the **test_data**.

Comparing Central Tendencies

1. Across Gender:

Code:

```
#central tendencies

score_summary <- final_data %>%
  group_by(gender) %>%
  summarise(
    Mean_Score = mean(final_exam_score, na.rm = TRUE),
    Median_Score = median(final_exam_score, na.rm = TRUE),
    Mode_Score = as.numeric(names(which.max(table(final_exam_score)))),
    .groups = "drop"
  )
print(score_summary)
```

Output:

```
> print(score_summary)
# A tibble: 2 × 4
  gender Mean_Score Median_Score Mode_Score
   <dbl>      <dbl>        <dbl>      <dbl>
1      0      0.468        0.351          1
2      1      0.466        0.344          1
>
```

The mean, median, and mode of **final_exam_score** were calculated for each **gender** group. The values are nearly identical between the two groups, suggesting gender has little to no association with final exam performance in this dataset.

2. Across Internet Access:

Code:

```
160
161 #internet access
162 score_summary_internet <- final_data %>%
163   group_by(internet_access) %>%
164   summarise(
165     Mean_Score = mean(final_exam_score, na.rm = TRUE),
166     Median_Score = median(final_exam_score, na.rm = TRUE),
167     Mode_Score = as.numeric(names(which.max(table(final_exam_score)))),
168     .groups = "drop"
169   )
170 print(score_summary_internet)
```

Output:

```
> print(score_summary_internet)
# A tibble: 2 × 4
  internet_access Mean_Score Median_Score Mode_Score
            <dbl>      <dbl>        <dbl>      <dbl>
1               0      0.337        0.255      0.246
2               1      0.501        0.435      1
>
```

Grouping by **internet_access** reveals a clearer difference: students without internet access have a notably lower mean (0.337) and median (0.255) score compared to students with access (mean 0.501, median 0.435). This suggests **internet_access** is more meaningfully associated with exam performance than **gender**.

Comparing Spread Across Gender

Code:

```r
#cmp spread across group
score_spread_summary <- final_data %>%
  group_by(gender) %>%
  summarise(
    Range_Score = max(final_exam_score, na.rm = TRUE) - min(final_exam_score, na.rm = TRUE),
    IQR_Score = IQR(final_exam_score, na.rm = TRUE),
    Variance_Score = var(final_exam_score, na.rm = TRUE),
    SD_Score = sd(final_exam_score, na.rm = TRUE),
    .groups = "drop"
  )
print(score_spread_summary)
```

Output:

```
> print(score_spread_summary)
# A tibble: 2 × 5
  gender Range_Score IQR_Score Variance_Score SD_Score
   <dbl>       <dbl>     <dbl>          <dbl>    <dbl>
1      0           1     0.390         0.0676    0.260
2      1       0.919     0.403         0.0693    0.263
>
```

The spread measures (range, IQR, variance, standard deviation) of **final_exam_score** are nearly identical across both gender groups. This reinforces the earlier finding - gender does not meaningfully affect how much scores vary, only marginally where they center, and even that difference is minor.

Conclusion

The dataset was verified for completeness and subsequently preprocessed by encoding categorical variables, eliminating outliers through the IQR method, filtering and normalizing with min-max normalization, and addressing the imbalanced Pass/Fail target via over and under sampling. The balanced dataset was split into 80% training and 20% testing sets. Comparisons of test scores revealed that gender did not correlate with final exam results, while internet access did. Individuals with access achieved notably higher scores on average and at the median.